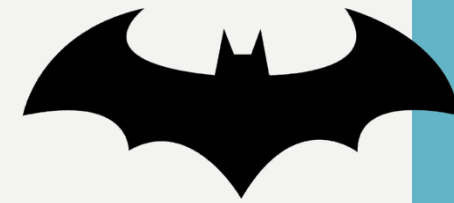




TP4 - OPÉRATION ARKHAM CITADEL

PROTOCOLES D'AUTHENTIFICATION

CONTEXTE



Le système central de la Batcave (développé lors des précédents TP) est fonctionnel, mais face aux menaces grandissantes du Joker et du Sphinx, son architecture réseau et applicative doit être blindée.

En tant que développeur de Batman, votre objectif est de faire passer l'API en mode **Zéro-Confiance** en implémentant des en-têtes de sécurité, en renforçant le cycle de vie de vos tokens et en érigeant une barrière **Double Facteur (MFA/2FA)** inviolable !

Ce TP s'effectue en autonomie sur votre propre code déjà existant. Vous devez travailler sur votre dépôt Git actuel en créant une nouvelle branche nommée `feature/mfa`, par exemple.





INSTRUCTIONS

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

ÉTAPE 0 : INSTALLATION DES DÉPENDANCES

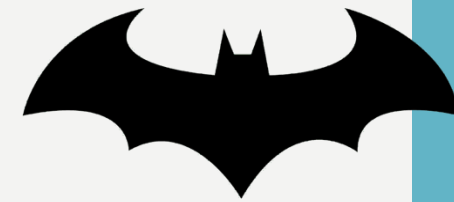
Avant de commencer à développer, vous devez installer les modules cryptographiques et de sécurité nécessaires.

Installez les dépendances suivantes :



```
npm install helmet qrcode @otplib/preset-v11
```

ÉTAPE 1 : INJECTION D'HELMET



Pour éviter que le navigateur de la Batcave ne divulgue des informations ou n'exécute des scripts malveillants, vous devez **configurer des en-têtes HTTP de sécurité**.

Dans votre point d'entrée serveur `server.js`, importez le module [helmet](#) et intégrez-le comme middleware global au début du cycle de requêtes de votre application.

Démarrez votre serveur, ouvrez les outils de développement de votre navigateur (F12), allez dans l'onglet **Réseau**, rechargez la page et inspectez les en-têtes de réponse.

Vous devriez voir apparaître une dizaine de nouveaux en-têtes de protection, notamment `Content-Security-Policy`, `X-Content-Type-Options: nosniff` ou encore `X-Frame-Options: SAMEORIGIN`.

ÉTAPE 2 : SÉCURITÉ DES COOKIES

Au TP précédent, vous avez mis en place deux cookies distincts : un jeton d'accès (JWT éphémère) et un jeton de rafraichissement (stocké en base). Nous allons nous assurer de verrouiller ces cookies pour bloquer les attaques par rejeu (ou *Replay Attacks*) et les interceptions MitM.

Ainsi, si ça n'est pas déjà fait, mettez à jour la configuration de la distribution de vos cookies (dans les routes `/login` et `/refresh`) en y ajoutant les directives restrictives indispensables contre les attaques XSS et CSRF : `httpOnly: true, sameSite: 'strict'`.

Ajustez la durée de vie du jeton d'accès à **15 minutes maximum**.

Assurez-vous que votre base de données possède bien une table pour suivre la validité des jetons de rafraichissement associés à l'identité de l'utilisateur afin de pouvoir révoquer une session si nécessaire.

ÉTAPE 3 : ENRÔLEMENT 2FA

L'accès aux dossiers confidentiels de Gotham exige désormais que chaque utilisateur possède une clé TOTP (pour *Time-Based One-Time Password*) sur son smartphone !

L'activation du 2FA se fait en deux temps (scénario asynchrone) pour éviter qu'un utilisateur ne se verrouille dehors.

Modifiez le modèle de la table `users` dans votre base de données SQLite pour y ajouter deux champs :

- `two_factor_secret` (`TEXT`, accepte `NULL`)
- `two_factor_enabled` (`INTEGER`, valeur par défaut `0`)

ÉTAPE 4 : INITIALISATION DE LA 2FA

Cette route est accessible uniquement si l'utilisateur est déjà connecté avec son premier facteur.

1. Générez une clé secrète cryptographique unique à l'aide de la documentation de la librairie [otplib](#) ;
2. Créez une URI standardisée de type `otpauth://` contenant le nom de votre application (`Batcave`, par exemple), l'identifiant de l'utilisateur, et le secret tout juste généré ;
3. Sauvegardez temporairement ce secret dans la ligne de l'utilisateur en BDD, mais **laissez la colonne** `two_factor_enabled` à `0` ;
4. Utilisez le module [qrcode](#) pour transformer l'URI `otpauth://` en une image encodée en **Base64** ;
5. Renvoyez au frontend l'image du QR Code ainsi que le secret en clair (pour affichage de secours).

ÉTAPE 5 : CONFIRMATION DE LA 2FA

L'utilisateur va donc voir le QR code s'afficher, le scanner avec son smartphone (Google Authenticator ou autre) et saisir le premier code à 6 chiffres généré par l'application pour prouver que la synchronisation fonctionne :

1. Récupérez le code à 6 chiffres et le nom d'utilisateur (ou e-mail) envoyés par le client ;
2. Récupérez le secret stocké en attente dans votre base de données ;
3. À l'aide de la méthode de vérification de la librairie *otplib*, confrontez le code reçu avec le secret et le timestamp du serveur ;
4. Si le code est invalide ou expiré, renvoyez une erreur `401 Unauthorized` et avortez l'activation ;
5. Si le code est valide, mettez à jour la base en passant `two_factor_enabled` à `1`. Renvoyez un message de succès.

ÉTAPE 6 : ZÉRO-CONFIANCE

Maintenant que les comptes peuvent être protégés par la 2FA, vous devez intercepter et durcir le point d'entrée principal de votre API : la route `/login`.

Modifiez la logique de votre contrôleur de connexion pour valider l'existence de l'utilisateur et comparez le mot de passe reçu avec le hachage en base. Si c'est incorrect, renvoyez une erreur `401`.

Si les identifiants sont corrects, regardez la valeur de la colonne `two_factor_enabled` en base :

- Si l'utilisateur n'a pas encore activé sa 2FA (`two_factor_enabled == 0`), le système rejette l'accès en renvoyant un code `403 Forbidden` avec un message obligeant l'activation de la 2FA. ⚠ **Aucun jeton ne doit être distribué !**
- Si la 2FA est active (`two_factor_enabled == 1`), interrompez la fonction. Ne distribuez pas encore le JWT d'accès. Renvoyez une réponse JSON structurée sous cette forme :

```

Réponse JSON

{
  "requires2FA": true,
  "message": "Étape 1 validée. Veuillez fournir votre code TOTP.",
  "username": "batman"
}

```

[Code à copier](#)

ÉTAPE 7 : VALIDATION FINALE

Il s'agit du second verrou de ce protocole d'authentification. Si le frontend reçoit le drapeau `requires2FA: true`, il affiche un champ de saisie à 6 chiffres qui pointe vers la route `/api/verify-2fa`:

1. Créez la route `/api/verify-2fa` ;
2. Récupérez le nom d'utilisateur (ou e-mail) et le code TOTP saisis ;
3. Récupérez son `two_factor_secret` en base ;
4. Utilisez le validateur d'*otplib* pour vérifier la concordance mathématique du code par rapport au temps actuel ;
5. ❌ Si le code échoue, renvoyez une erreur `401 Unauthorized` ;
6. 🎉 Si le code est valide, l'authentification est complète. Générez le jeton d'accès (15m) et le jeton de rafraichissement, placez-les dans vos cookies sécurisés et validez la connexion.



POUR ALLER PLUS LOIN

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

CHALLENGE 1 : DÉTECTION DE VOL DE JETONS

Seulement pour les étudiants ayant terminé les phases précédentes !

Au TP précédent, vous avez mis en place le renouvellement des jetons avec suppression de l'ancien jeton de rafraichissement en base. Que se passe-t-il si un pirate réussit à intercepter un refresh_token avant qu'il ne soit consommé, et tente de l'utiliser ? Si l'utilisateur légitime l'utilise juste après, le serveur va simplement lui dire que le jeton n'existe plus. Mais le pirate, lui, a réussi à obtenir un jeton d'accès valide pendant ce temps !

👉 Modifiez votre base de données et votre route `/refresh` pour implémenter une politique de **révocation automatique par jeu** :

1. Créez un historique ou ajoutez un statut `is_used` (ou une table `invalidated_tokens`) pour conserver la trace des anciens jetons de rafraichissement pendant leur durée de validité théorique ;
2. Si la route `/refresh` reçoit un jeton de rafraichissement **qui a déjà été utilisé dans le passé**, le serveur doit comprendre qu'une anomalie (ou un vol) a eu lieu ;
3. Le serveur doit instantanément **révoquer et supprimer TOUS les jetons de rafraichissement actifs** associés à cet utilisateur en base. Batman et le pirate seront immédiatement déconnectés et forcés de se réauthentifier via la MFA.

CHALLENGE 2 : GESTION FINE DES PRIVILÈGES

Seulement pour les étudiants ayant terminé les phases précédentes !

Dans le protocole [OAuth2](#), un jeton ne donne pas des droits d'accès globaux à un utilisateur, mais des autorisations spécifiques appelées des **scopes** (ex. `read:profile`, `write:orders`). Nous allons adapter le système de la Batcave pour introduire ce concept.

👉 Imaginons que Batman souhaite déléguer un accès technique à Nightwing ou à Oracle via un jeton mais avec des droits restreints :

1. Modifiez la structure de votre payload JWT (lors de sa génération dans `/verify-2fa` ou `/refresh`) pour y intégrer un tableau de chaînes de caractères nommé `scopes` :
 - Exemple pour Batman : `scopes: ["computers:admin", "batmobile:control", "armory:weapons"]`
 - Exemple pour un allié : `scopes: ["computers:read", "batmobile:status"]`
2. Créez un middleware Express réutilisable nommé `checkScope(requiredScope)` qui intercepte la requête **après** la validation du JWT ;
3. Ce middleware doit vérifier si le scope requis pour l'action est explicitement présent dans le payload du jeton de l'utilisateur. Si le scope est manquant, l'API doit renvoyer un statut `403 Forbidden` (même si le JWT est mathématiquement valide et non expiré) ;
4. Appliquez ce middleware sur vos routes existantes : l'accès aux commandes de la Batmobile ou à l'armurerie doit exiger des privilèges différents de la simple lecture du profil.

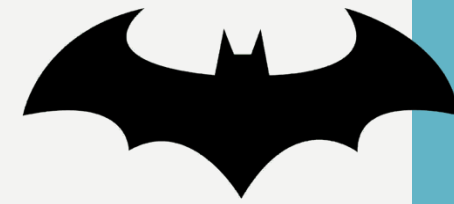


MODALITÉS D'ÉVALUATION ET CONTRÔLE CONTINU

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

« CODE CHECK »



Ce projet filé ne sera pas simplement noté sur votre dépôt Git.

👉 Lors de la séance suivante, j'échangerai avec **chaque étudiant** (non rencontré) **en face-à-face** pendant 3 à 10 minutes sur le code et sa compréhension.

Les règles du jeu :

- Je pointerai une ligne de code que vous devrez m'expliquer précisément ;
- Attendez-vous à devoir justifier vos choix et être challengé sur votre maîtrise des concepts vus en cours.

❌ Si vous êtes incapable d'expliquer un bout de code présent sur dans votre dépôt ou si le vocabulaire technique utilisé à l'oral ne correspond pas à votre niveau supposé, la note en sera impactée.



DES QUESTIONS ?

CHEVALIER@CHRIS-FREELANCE.COM